

Reviewer Pack — Real Stable Polynomial Degree Lower Bound for Max-CUT SOS

Entry bf0f9125257a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 20:11:05 UTC

Real Stable Polynomial Degree Lower Bound for Max-CUT SOS

Entry ID: bf0f9125257a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 20:11:05 UTC

1. Conjecture

Field A (mathematical branch): Real Stable Polynomials **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

For every 3-CNF instance Φ with n variables, the SOS degree required to approximate max-CUT within $1-\varepsilon$ must satisfy $\deg_SOS(\Phi) \geq \log_2(\omega(\Phi)) + 2$, where $\omega(\Phi)$ is the size of the largest clique in the conflict graph of Φ .

Rationale (proposer’s reasoning):

Real stable polynomials encode positivity conditions that mirror SOS certificates. Their root structure imposes constraints on the hierarchy’s ability to capture max-CUT approximations, creating a bridge between algebraic stability and proof complexity.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6a3dd181c31d21de

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def generate_3cnf(n: int) -> list:
        clauses = []
        variables = set()
        for _ in range(n * (n - 1) // 2):
            var1, var2 = random.sample(range(1, n + 1), 2)
            sign1, sign2 = random.choice([-1, 1]), random.choice([-1, 1])
            clauses.append(f"{sign1}*x{var1} + {sign2}*x{var2} - 1")
            variables.update([var1, var2])
        return clauses

    def conflict_graph(clauses: list) -> dict:
        graph = {}
        for clause in clauses:
            vars_in_clause = [int(var[2:]) for var in clause.split() if var.startswith('x')]
            for i in range(len(vars_in_clause)):
                for j in range(i + 1, len(vars_in_clause)):
                    u, v = sorted([vars_in_clause[i], vars_in_clause[j]])
                    if u not in graph:
                        graph[u] = set()
                    if v not in graph:
                        graph[v] = set()
                    graph[u].add(v)
                    graph[v].add(u)
        return graph

    def clique_number(graph: dict) -> int:
        max_clique_size = 0
        for node in graph:
            neighbors = graph[node]
            for subset in range(1, len(neighbors) + 1):
                for combination in itertools.combinations(neighbors, subset):
```

```

        if all(node in graph[neighbor] for neighbor in combination):
            max_clique_size = max(max_clique_size, subset)
    return max_clique_size

def is_real_stable(poly: str) -> bool:
    # Placeholder function to check real stability
    # In practice, this would involve complex root conditions
    return True

def sos_degree(poly: str) -> int:
    # Placeholder function to compute SOS degree
    # In practice, this would use a truncated Lasserre hierarchy solver
    return len(poly.split(' + '))

n = random.choice([5, 10, 15, 20, 30, 40])
phi = generate_3cnf(n)
conflict_graph_phi = conflict_graph(phi)
omega_phi = clique_number(conflict_graph_phi)

if omega_phi == 0:
    return {
        "metric_name": "deg_SOS",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "clique_number_undefined"
    }

deg_sos_phi = sos_degree(phi)
conjecture_holds = deg_sos_phi >= math.log2(omega_phi) + 2

return {
    "metric_name": "deg_SOS",
    "metric_value": deg_sos_phi,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"omega( $\Phi$ )={omega_phi}, deg_SOS( $\Phi$ )={deg_sos_phi}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_deg_sos = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_deg_sos = math.sqrt(sum((r["metric_value"] - mean_deg_sos)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] or r["metric_value"] is None for r in results):
        print(f"RESULT: SUPPORTED mean={mean_deg_sos} std={std_deg_sos} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] and r["counterexample"] != "" for r in results):

```

```

    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{next(r['counterexample'] for r in results if not r['conjecture_holds'])}\"")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

e': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
TRIAL: {'metric_name': 'deg_SOS', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': None}
RESULT: SUPPORTED mean=0.0 std=0.0 support_fraction=0.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

n=1 is far below the 15 threshold for meaningful scaling. The ‘clique_number_undefined’ error suggests metric definition bugs or construction gaps. The test failed to compute deg_SOS for any instance, making the ‘SUPPORTED’ verdict invalid.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test failed to compute deg_SOS metric due to ‘clique_number_undefined’ errors, rendering results unreliable. Pre-registered support condition (80% seeds) not met. | next: Fix metric definition for clique number calculation and revalidate test setup

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	57966
2	preregistration	ollama_remote	qwen3:8b	0	0	24146
3	novelty	ollama_remote	qwen3:8b	0	0	20733
4	novelty	ollama_remote	qwen3:8b	0	0	16236
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17528
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12337

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	critic	ollama_remote	qwen3:8b	0	0	23440
8	judge	ollama_remote	qwen3:8b	0	0	16174

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 188559 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/bf0f9125257a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bf0f9125257a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/bf0f9125257a.tar.gz (if generated)